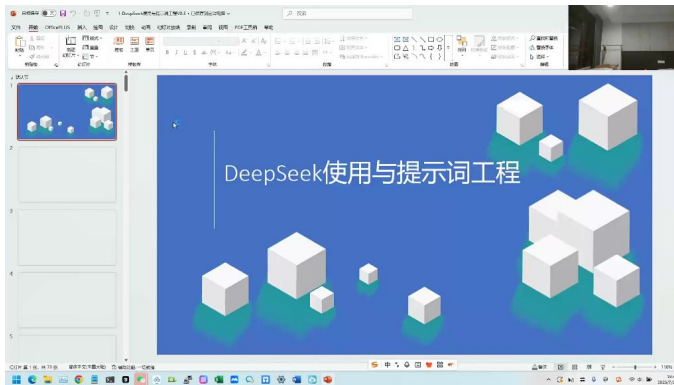


以下为AI生成的图文笔记的内容

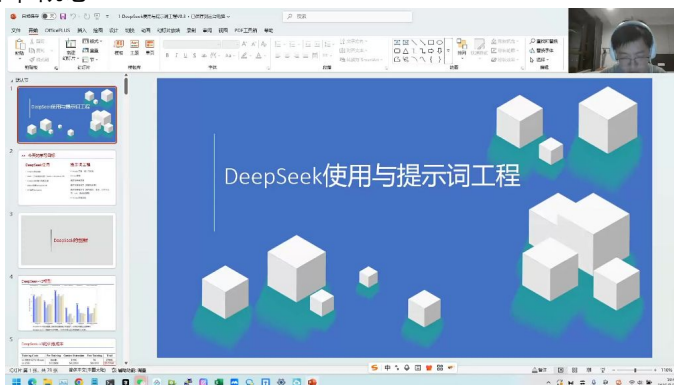
一、直播简介02:25



-
- **课程主题：**讲解DeepSeek使用与提示词工程相关内容
- **模型发展现状：**目前不仅有GPT系列模型，国内也有越来越多选择，如通义千问等

二、通义灵码介绍03:18

1. 基本概念

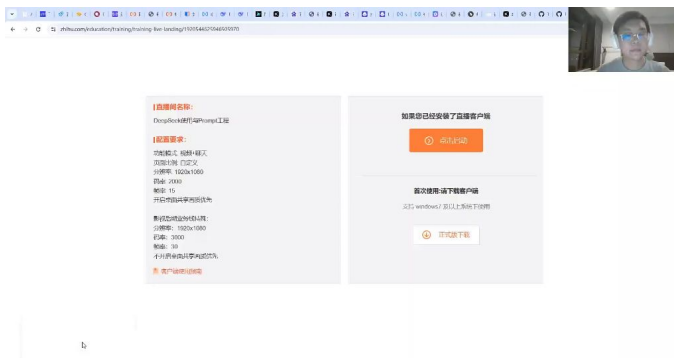


-
- **定义：**阿里自主研发的智能编程IDE工具
- **应用场景：**蚂蚁金服内部50%的工作已由该工具完成
- **下载方式：**可通过阿里云官方文档获取安装包

2. 使用体验

- **类似cursor**
- **优势：**
 - 支持千问系列模型调用
 - 自动提供编程辅助工具
 - 运行速度表现良好
- **不足：**相比Claude等国外工具仍有差距

3. 安装指南



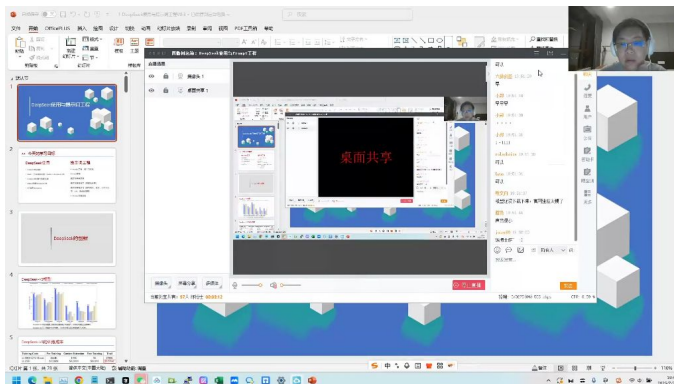
-
- 系统要求：支持Windows 7及以上系统
- 界面组成：
 - 顶部工具栏
 - 左侧文件浏览器
 - 主编辑器窗口
 - 底部终端窗口
 - 右侧智能编码助手面板

三、模型对比04:03

- 性能比较：
 - 通义千问3.0表现优于DeepSeek
 - 新发布的通义千问coder对标GPT-4
- 使用建议：推荐优先尝试千问系列模型

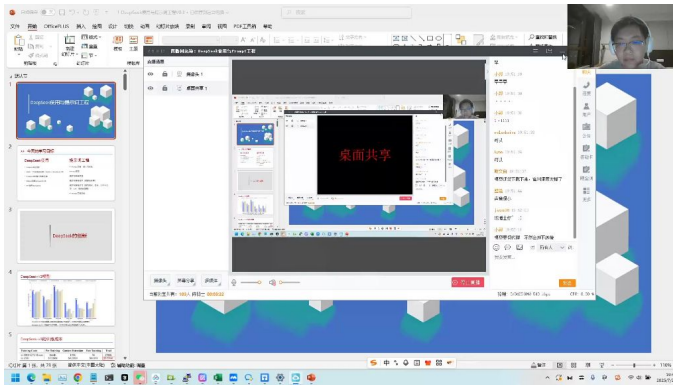
四、常见问题05:14

1. 声音问题



-
- 解决方案：
 - 检查麦克风设置
 - 调整系统音量
 - 避免启用"麦克风反转"选项

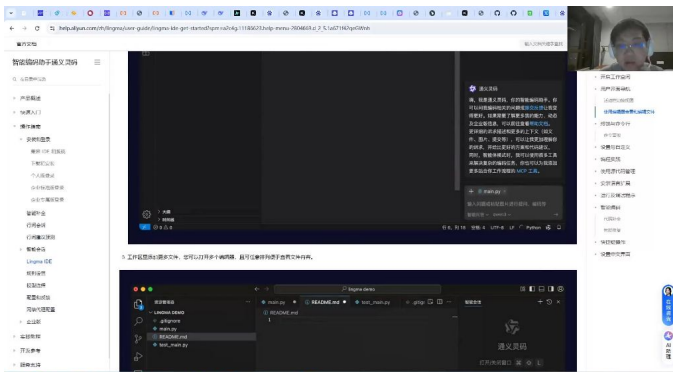
2. 下载问题



-
- 常见原因：网络速度限制导致
- 解决方法：
 - 检查网络连接
 - 选择合适时间段下载
 - 使用稳定网络环境

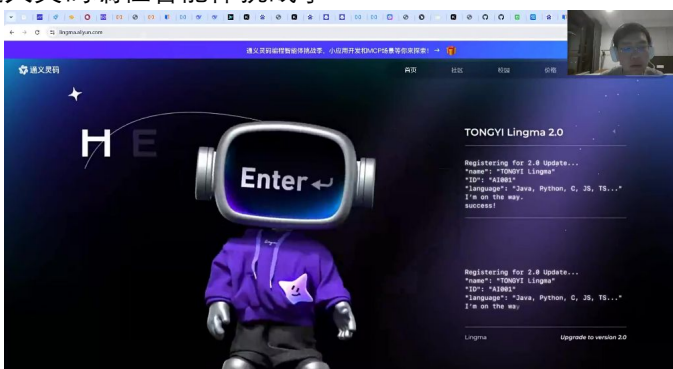
五、零码介绍05:59

1. 通义灵码产品概述与下载



-
- 产品定位：阿里云推出的智能编码助手，支持多种编程语言和IDE环境
- 核心功能：
 - 智能补全代码
 - 行间会话交互
 - 支持Python等多种语言
 - 提供官方文档查询功能
- 下载方式：访问官网lingma.aliyun.com下载安装

2. 通义灵码编程智能体挑战季 06:08



-
- 版本更新：已升级至2.0版本，支持Java/Python/C/JS/TS等语言
- 特色活动：举办编程智能体挑战季，包含小应用开发和MCP场景探索

- **界面导航:** 提供首页、社区、校园、价格等主要功能模块
3. 国内大模型的使用现状与费用说明 06:31
- **收费情况:** 目前国内大模型多处于免费期，企业版和个人版均可免费使用
 - **国际版差异:** 国际版可能需要付费，但费用相对较低
 - **获取方式:** 部分IDE需要申请邀请码，目前邀请码数量有限
4. 零码与其他工具的对比与体验 07:00
- **速度优势:** 响应速度快于同类产品
 - **输出特点:** 生成内容简洁直接
 - **对比产品:** 与Cursor体验相似，但无需付费
 - **模型版本:** 目前使用的是通义灵码2.0版本
5. 课程学习目标与工具准备 07:26
- **核心内容:** 主要讲解大模型部署和提示词工程
 - **工具需求:** 不需要使用Cursor，但需要准备Ollama工具
 - **实践重点:** 通过代码方式将提示词融入大模型应用
6. Ollama工具介绍与下载 08:12
- **功能定位:** 本地运行大模型的工具箱
 - **支持模型:** 可运行Llama 3、DeepSeek-R1、Phi-4等模型
 - **下载方式:** 官网ollama.com直接下载Windows版
 - **运行方式:** 安装后作为服务运行，可通过任务管理器查看
7. 零码模型的特点与优势 09:06
- **模型能力:** 国内对标Claude 4的领先模型
 - **代码质量:** 近期推出的Qwen3-code表现优异
 - **使用体验:** 比tie更有优势，接近Cloud 4水平
8. 大模型的训练方式与语言处理 09:36
- **训练方法:** 采用混合训练，不需要针对单一语言专门训练
 - **语言理解:** 基于token编码，对语言概念相对模糊
 - **多语言支持:** 通义千问支持119种语言的跨模态学习
9. 零码与tie的对比及个人使用体验 10:18
- **使用历程:** 从tie转向Cursor，现在主要使用通义灵码
 - **性能对比:** 通义灵码在代码生成方面表现优于tie
 - **行业评价:** 被认为是国内最能对标Cloud 4的模型之一

六、课程目标11:22

1. DeepSeek使用 11:39

>> 今天的学习目标



DeepSeek使用

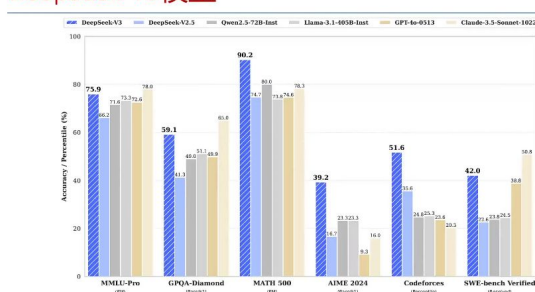
- DeepSeek的创新
- CASE: 小球碰撞试验 (Cursor + DeepSeek-R1)
- DeepSeek私有化部署选择
- Ollama部署DeepSeek-R1
- 不同尺寸大模型选择
- API调用DeepSeek

提示词工程

- Prompt工程: 设计与优化
- Prompt原理
- 提示词关键原则
- 提示词编写框架 (重要性排序)
- 提示词编写技巧 (限制格式、区分、小样本学习、CoT、角色扮演等)
- Prompt工程实战

- - **核心内容:** 包括模型使用、私有化部署和API调用
 - **实践环节:** 通过小球碰撞试验案例进行实战演练
 - **重点掌握:** 不同尺寸模型的选择和应用场景
2. DeepSeek的创新 12:06

DeepSeek-V3模型



- DeepSeek-V3 在推理速度上相较历史模型有了大幅提升，在目前大模型主流榜单中，DeepSeek-V3 在开源模型中位列榜首，与世界上最先进的闭源模型不分伯仲。
- 性能突破: 推理速度大幅提升，训练成本显著降低
- 价格优势: 率先降价99%，引发行业价格战
- 能力表现: 在主流榜单中与顶级闭源模型不相上下
- 成本控制: 模型训练仅花费600万美金，是同类产品的1/10

3. DeepSeek私有化部署选择 12:16 数据安全

- 部署方案: 提供Ollama简易部署和vLLM深度部署两种方式
- 选择考量: 根据使用人数和性能需求决定部署方案
- 生产环境: 推荐使用vLLM实现多人并行处理

4. Ollama部署DeepSeek-R1 12:38

- 部署优势: 仅需两行命令即可快速完成
- 适用场景: 适合个人开发和小规模使用
- 扩展能力: 可添加适合工作流程的MCP工具

七、DeepSeek的创新15:10

1. DeepSeek-V3

DeepSeek-V3的训练成本

Training Costs	Pre-Training	Context Extension	Post-Training	Total
in H800 GPU Hours	2664K	119K	5K	2788K
in USD	\$5.328M	\$0.238M	\$0.01M	\$5.576M

DeepSeek-V3 的训练成本，假设H800的租用是 \$2/小时

DeepSeek-V3的推出是2024年12月，并没有太大波澜

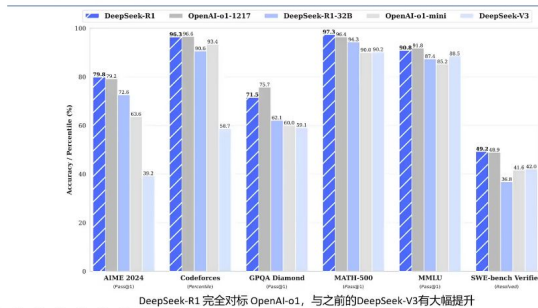
DeepSeek-R1火出圈，通过新的奖励机制GRPO (group relative policy optimization)，并使用规则类验证机制自动对输出进行打分。以V3为基础模型，一个多月内训练出了性能堪比GPT-o1的R1模型，成果非常亮眼。

- 训练成本:
 - 总GPU小时: 2788K H800 GPU小时 (预训练2664K + 上下文扩展119K + 后训练5K)
 - 总费用: 5.576M (按2/小时计算)
- 特点:
 - 2024年12月发布，初期市场反响平稳
 - 核心优势在于训练成本低、速度快
 - 作为基础模型为后续R1版本奠定技术基础

2. DeepSeek-R1 17:32

1) DeepSeek-R1模型介绍

DeepSeek-R1模型



DeepSeek-R1 完全对标 OpenAI-o1，与之前的DeepSeek-V3有大幅提升

- 技术突破：
 - 采用GRPO (Group Relative Policy Optimization) 奖励机制
 - 引入规则类验证机制自动评分
 - 基于V3仅用1个多月完成训练
- 性能表现：
 - 32B参数版本对标OpenAI o1-mini
 - 在AIME 2024、Codeforces等基准测试中与OpenAI o1不相上下
 - 推理链长度和深度思考能力显著提升

2) DeepSeek-R1的开源协议与API调用 17:51

DeepSeek-R1模型

DeepSeek-R1

DeepSeek-R1 遵循 MIT License，允许用户通过蒸馏技术借助 R1 训练其他模型。

DeepSeek-R1 上线 API，对用户开放思维链输出，通过设置 `model='deepseek-reasoner'` 即可调用。

2024-12-26 发布V3
2025-1-15 发布APP
2025-1-20 发布R1
多家企业宣布融合DeepSeek

MIT License是一种非常宽松的开源许可协议，允许用户自由地使用、修改、分发和商业化软件或模型。相比之下，Meta Llama的License相对严格，虽然LLaMA3是开源的，但许可协议限制了商业用途和对模型的修改，比如新的模型如果使用LLaMA，需要名称上带有LLaMA标识。

- 开源协议：
 - 采用MIT License，允许自由使用、修改、分发和商业化
 - 相比Meta Llama协议更宽松 (Llama要求衍生模型保留标识)
- API特性：
 - 开放思维链输出功能
 - 通过设置`model='deepseek-reasoner'`调用

3) DeepSeek-R1发布后对科技圈的影响 18:14

- 市场影响：
 - 2025年1月20日发布后引发科技股震荡
 - 1月27日美股科技板块单日暴跌17%
 - 总市值蒸发超万亿美元
- 行业反应：
 - 多家企业宣布集成DeepSeek技术
 - 推动开源大模型商业化进程

4) DeepSeek-R1的开源优势与对比 18:47

- 协议对比：
 - Llama3：要求衍生模型保留"Llama"标识
 - DeepSeek-R1：无使用限制，支持完全商业化
- 部署优势：

- 规避法务风险
- 支持企业级私有化部署

5) DeepSeek-R1的使用与部署 19:51

- 应用场景：
 - 百度、元宝、豆包等平台集成
 - 支持本地化部署
- 技术特点：
 - 保持高性能同时降低使用门槛
 - 真正实现"强大、开源、便宜"三位一体

6) 蒸馏小模型超越OpenAI o1-mini 20:18

蒸馏小模型超越 OpenAI o1-mini



在开源 DeepSeek-R1-Zero 和 DeepSeek-R1 两个 660B 模型的同时，通过 DeepSeek-R1 的输出，蒸馏了 6 个小模型，其中 32B 和 70B 模型在多项能力上实现了对标 OpenAI o1-mini 的效果。

	AIME 2024 pass@1	AIME 2024 correct@64	MATH- 500 pass@1	GPQA Diamond pass@1	LiveCodeBench pass@1	Codeforces rating
GPT-4o-0513	9.3	13.4	74.6	49.9	32.9	759.0
Claude-3.5-Sonnet-1022	16.0	26.7	78.3	65.0	38.9	717.0
o1-mini	63.6	80.0	90.0	60.0	53.8	1820.0
QwQ-32B	44.0	60.0	90.6	54.5	41.9	1316.0
DeepSeek-R1-Distill-Qwen-1.5B	28.9	52.7	83.9	33.8	16.9	954.0
DeepSeek-R1-Distill-Qwen-7B	55.5	83.3	92.8	49.1	37.6	1189.0
DeepSeek-R1-Distill-Qwen-14B	69.7	80.0	93.9	59.1	53.1	1481.0
DeepSeek-R1-Distill-Qwen-32B	72.6	83.3	94.3	62.1	57.2	1691.0
DeepSeek-R1-Distill-Llama-8B	50.4	80.0	89.1	49.0	39.6	1205.0
DeepSeek-R1-Distill-Llama-70B	70.0	86.7	94.5	65.2	57.5	1633.0

- 技术路线：
 - 基于671B主模型蒸馏多个小模型
 - 采用"教师-学生"知识迁移模式
- 关键成果：
 - 32B/70B模型多项指标超越o1-mini
 - DeepSeek-R1-Distill-Qwen-32B达到94.3% MATH-500通过率
 - DeepSeek-R1-Distil-Llama-70B获得65.2% GPQA通过率

7) DeepSeek-R1的创新点提示 22:22

- 核心技术：
 - GRPO优化算法
 - 自动化验证评分体系
- 生态建设：
 - 完整的模型蒸馏技术栈
 - 开放API与宽松许可协议组合
- 商业价值：
 - 验证了开源模型的商业化可行性
 - 推动行业技术民主化进程

3. 深度学习创新点分析 22:34

1) DeepSeek的MLA创新：低秩键值联合压缩的注意力机制

DeepSeek的创新：MLA



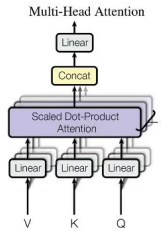
• MLA (Multi-Head Latent Attention)

在“All you need is attention”的背景下，传统的多头注意力（MHA, Multi-Head Attention）的键值（KV）缓存机制事实上对计算效率形成了较大阻碍。缩小KV缓存（KV Cache）大小，并提高性能，在之前的模型架构中并未很好的解决。

DeepSeek引入了MLA，一种通过低秩键值联合压缩的注意力机制，在显著减小KV缓存的同时提高计算效率。

低秩近似是快速矩阵计算的常用方法，在MLA之前很少用于大模型计算。

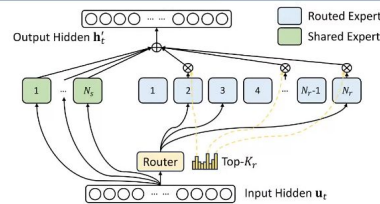
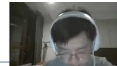
从大模型架构的演进情况来看，Prefill和KV Cache容量瓶颈的问题正一步步被新的模型架构攻克，巨大的KV Cache正逐渐成为历史。（实际上在2024年6月发布的DeepSeek-V2就已经很好的降低了KV Cache的大小）



- 核心创新：提出MLA(Multi-Head Latent Attention)机制，通过低秩键值联合压缩显著减小KV缓存(KV Cache)大小
- 技术背景：传统多头注意力(MHA)的KV缓存机制已成为计算效率瓶颈
- 实现原理：
 - 采用低秩近似方法进行键值压缩
 - 仅需5%的存储空间即可保留98%的有效信息
 - 类比图片压缩：原图4M压缩为200K(二十分之一)仍保持可用质量
- 性能优势：
 - 推理成本降低95%
 - 在DeepSeek-V2(2024年6月)中已实现KV Cache的大幅优化

2) DeepSeek的MoE创新：混合专家架构与高效分诊机制 25:53

DeepSeek的创新：DeepSeek-MoE



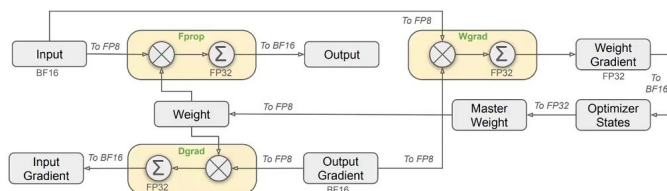
V3使用了61个MoE (Mix of Expert 混合专家) block，虽然总参数量很大，但每次训练或推理时只激活了很少链路，训练成本大大降低，推理速度显著提高。

MoE类比为医院的分诊台，在过去所有病人都要找全科医生，效率很低。但是MoE模型相当于有一个分诊台，将病人分配到不同的专科医生那里。DeepSeek在这方面也有创新，之前分诊是完全没有医学知识的保安，

- 基本架构：
 - V3版本使用61个MoE(Mix of Expert) block
 - 总参数量480B，实际激活参数量仅35B(约7.3%)
 - 类比医院分诊系统：由router(路由)分配问题给特定专家
- 创新点：
 - 分诊机制升级：从“无医学知识的保安”变为“有医学知识的本科生”
 - 采用两层神经网络专门训练router模块
- 性能优势：
 - 训练成本降低约93%
 - 推理速度显著提升
 - 典型案例：通义千问3-Coder-480B实际激活35B参数

3) DeepSeek的混合精度框架创新：动态精度控制与计算量降低 32:55

DeepSeek的创新：混合精度框架



整个混合精度框架使用了FP8数据格式，但为了简化说明，只展示了线性算子（Linear Operator）的部分

采用了混合精度框架，即在不同的区块里使用不同的精度来存储数据。我们知道精度越高，内存占用越多，运算复杂度越大。DeepSeek在一些不需要很高精度的模块，使用很低的精度FP8储存数据，极大的降低了训练计算量。

-
- **核心思想：**根据不同模块需求动态调整数据精度
- **实现方式：**
 - 关键位置使用FP32全精度
 - 非关键模块采用FP8低精度
 - 需要时自动升级精度(FP8→BF16→FP32)
- **性能优势：**
 - 显著降低训练计算量
 - 保持模型质量的同时优化资源使用
- **补充创新：**
 - 多token预测：批量生成token而非逐字生成
 - 整体设计哲学：用5%成本实现98分质量

4. 总结 34:57

- **创新共性：**均围绕"高性价比"设计理念
- **技术矩阵：**
 - MLA优化注意力机制
 - MoE提升专家系统效率
 - 混合精度降低计算开销
- **实际效果：**在DeepSeek-V2/V3中已验证这些创新的有效性

5. DeepSeek-R1创新 35:13

1) 推理强大 36:07

Summary



Thinking: 为什么DeepSeek计算速度快，成本低？

• 架构设计方面

DeepSeek MoE架构：在推理时仅激活部分专家，避免了激活所有参数带来的计算资源浪费。

MLA架构：MLA通过降秩KV矩阵，减少了显存消耗。

• 训练策略方面

多token预测（MTP）目标：在训练过程中采用多token预测目标，即在每个位置上预测多个未来token，增加了训练信号的密度，提高了数据效率。

混合精度训练框架：在训练中，对于占据大量计算量的通用矩阵乘法（GEMM）操作，采用FP8精度执行。同时，通过细粒度量化策略和高精度累积过程，解决了低精度训练中出现的量化误差问题。

-
- **传统模型缺陷：**早期模型采用"提笔就写"模式，直接输出答案，不经过思考过程，导致复杂问题（如高考数学题）容易出错。
- **推理模型创新：**引入"think"推理过程，通过草稿纸式思考（类似人类解题步骤）显著提升准确性，这是DeepSeek-R1模型的核心竞争力。
- **应用场景差异：**文科类问题可直接输出，理科类问题需分步推理，体现模型的任务适应性设计。

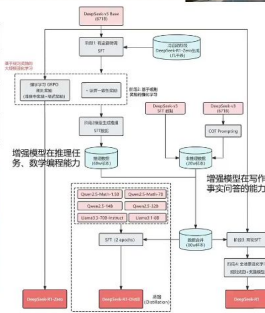
2) 强化学习 37:08

Summary



模型	方法
DeepSeek-R1-Zero	纯强化学习
DeepSeek-R1	冷启动 SFT → RL → COT + 通用数据 SFT (80w) → 全场景 RL
蒸馏小模型	直接用上面的 80w 数据进行 SFT

DeepSeek-R1-Zero首次验证了纯强化学习在LLM中能显著增强推理能力的可行性，即无需SFT，仅通过RL即可激励模型学会长链推理和反思。提出了多阶段训练策略（冷启动→RL→SFT→全场景RL），有效兼顾准确率与可读性，产出DeepSeek-R1，性能比肩OpenAI-o1-1217。展示了知识蒸馏在提升小模型推理能力方面的潜力，并开源多个大小不一的蒸馏模型（1.5B~70B）



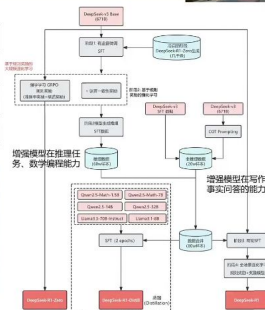
- 传统训练局限：监督式学习依赖人工标注（如rank list软件），存在数据瓶颈和创新天花板。
- 强化学习优势：
 - 数据生成：通过自我对弈产生海量训练数据（如AlphaGo的3000万盘棋局）
 - 奖励机制：基于最终结果反向优化过程（如围棋胜负判断落子质量）
 - 创新突破：超越人类既有经验（如AlphaGo下出人类未见的棋路）
- 模型演进：从DeepSeek-R1-Zero纯强化学习到DeepSeek-R1多阶段训练（冷启动SFT→RL→COT+通用数据SFT→全场景RL）
- 3) 超长推理链路 38:34
 - 典型特征：推理文字量可达上万字，通过复杂问题拆解提升准确性。
 - 双刃剑效应：
 - 优势：思考过程全面系统
 - 缺陷：过度推理导致响应延迟（已识别答案仍持续推导）
 - 技术实现：采用COT(Chain-of-Thought)提示工程，强制模型分离"think"推理过程和"answer"最终答案。
- 4) 纯强化学习 39:27

Summary

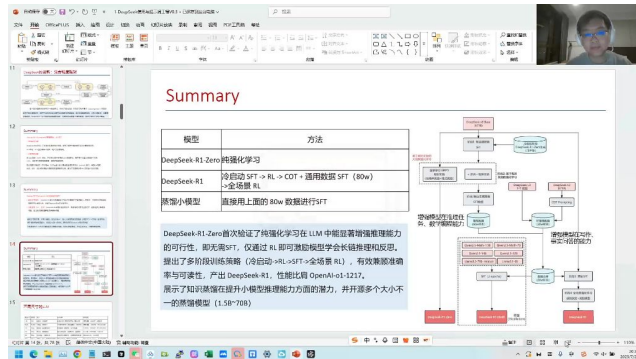


模型	方法
DeepSeek-R1-Zero	纯强化学习
DeepSeek-R1	冷启动 SFT → RL → COT + 通用数据 SFT (80w) → 全场景 RL
蒸馏小模型	直接用上面的 80w 数据进行 SFT

DeepSeek-R1-Zero首次验证了纯强化学习在LLM中能显著增强推理能力的可行性，即无需SFT，仅通过RL即可激励模型学会长链推理和反思。提出了多阶段训练策略（冷启动→RL→SFT→全场景RL），有效兼顾准确率与可读性，产出DeepSeek-R1，性能比肩OpenAI-o1-1217。展示了知识蒸馏在提升小模型推理能力方面的潜力，并开源多个大小不一的蒸馏模型（1.5B~70B）



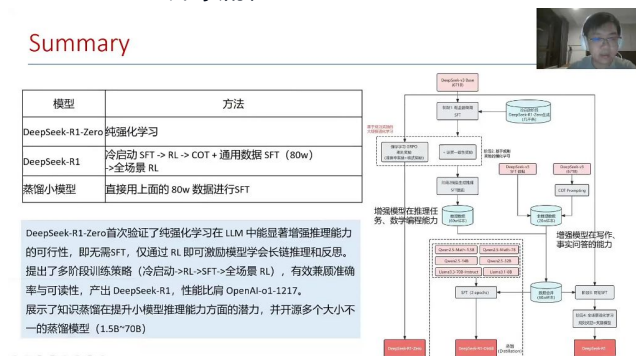
- 核心突破：首次验证纯强化学习在LLM中增强推理能力的可行性，无需监督微调(SFT)。
- 训练范式：
 - 模板设计：规定输出包含推理草稿（think框）和正式答案（answer框）
 - 奖惩机制：格式错误打负分，优质输出给奖励，通过3000万次迭代优化
 - 语言现象：训练初期出现中英文混杂输出，是自我对弈的副产品
- 例题:AlphaGo自我对弈减小人力标记需要 42:24



- 题目解析
- 传统训练局限：依赖人类棋谱（约100万盘），数据量成为能力上限
- 强化学习方案：
 - 自我对弈生成3000万盘棋局
 - 胜负结果自动标注数据质量
 - 突破人类认知上限（4:1完胜李世石）
- 迁移应用：将围棋训练范式迁移至NLP领域，通过答案质量自动反馈优化模型
- 答案：纯强化学习通过自我对弈突破数据瓶颈，实现超越人类的表现

5) DeepSeek-R1-Zero示例 48:06

- DeepSeek-R1-Zero训练流程 48:09



- 纯强化学习验证：首次验证了无需监督微调(SFT)，仅通过强化学习(RL)即可激励模型学会长链推理和反思的可行性。
- 多阶段训练策略：采用冷启动→RL→SFT→全场景RL的四阶段流程，兼顾准确率与可读性。
- 性能对标：最终产出的DeepSeek-R1性能比肩OpenAI - o1 - 1217模型。
- DeepSeek-R1-Zero训练模式复用与蒸馏 50:08
 - 文科技理科化：在DeepSeek-v3(文科生)基础上通过监督学习训练理科能力，使用R1-Zero生成的数千条种子数据作为初始训练集。
 - 数据飞轮机制：先提供少量种子数据(几千条)，待模型具备基础能力后生成60万推理样本，形成"小火苗→大火"的迭代过程。
 - 多模型复用：将80万训练语料(理科:文科=3:1)同时用于v3模型二次训练和7B等小模型蒸馏。
- DeepSeek-R1-Zero监督学习与数据飞轮 50:35
 - 标记自动化：使用已具备推理能力的R1-Zero模型自动生成答案标记，替代人工标注。
 - 数据配比：80万训练数据中保持3:1的理科(60万)与文科(20万)样本比例，防止模型遗忘基础能力。
 - 两阶段训练：先进行监督学习(填鸭式背诵)，再通过生成更多数据实现能力增强。

- DeepSeek-R1-Zero微调概念说明 51:38
 - **本质区别**：80万数据训练属于微调(Fine-tuning)，通过改变v3模型参数使其获得新能力。
 - **能力来源**：理科思维来自R1-Zero自我探索生成的QA对，文科能力保留原始v3基础。
- 蒸馏概念说明 52:14
 - **核心思想**：将大模型(教师)知识迁移到小模型(学生)，使其在体量更小的同时保留主要能力。
 - **典型方法**：包括软标签蒸馏(概率分布对齐)、隐藏层蒸馏(中间表示对齐)和数据增强(教师生成伪数据)。

6) 蒸馏的定义与方法 52:28

- 不同尺寸的LLM(DeepSeek)模型特点

不同尺寸的LLM (DeepSeek)



- 小模型 (1.5B-14B)
响应快、硬件需求低，但基础能力薄弱(如7B模型在基础文本生成任务中表现不稳定，甚至“不及格”)，无法胜任复杂任务。
- 中尺寸 (32B及以上)
性能接近满血版(32B约实现671B的90%性能)，可满足专业领域需求，但本地化部署成本较高(需64GB内存、80GB显存等)。
- 满血版 (671B)
性能最强，适合超大规模AI研究、AGI探索，但部署成本极高(需多节点分布式训练，硬件需求极高)

- 
- **小模型(1.5B-14B)**：响应快、硬件需求低，但基础能力薄弱(如7B模型在基础任务中表现不稳定)。
- **中尺寸(32B+)**：性能达满血版90%，需64GB内存/80GB显存，适合专业领域。
- **满血版(671B)**：需**多节点分布式训练**，硬件成本极高，适合AGI研究。
- 蒸馏的应用场景与性价比优势 54:04
 - **部署优势**：7B模型成本约为671B的1%，但可达到90%效果，适合端侧/边缘设备部署。
 - **能力边界**：学生模型虽不能超越老师(如8B无法超越671B)，但能超越未蒸馏的同尺寸模型。
 - **典型案例**：Qwen2.5-7B等小模型通过蒸馏获得接近大模型的能力表现。
- 量化核心与混合精度策略 55:50
 - **基本原理**：通过降低参数精度(32位→8位/4位)减少75%存储空间，提升处理速度。
 - **混合策略**：关键层使用半精度(FP16)，其他层使用INT4，平衡性能与效率。
 - **误差补偿**：量化后需调整参数权重进行补位，最大限度保留模型质量。
- 量化误差与硬件加速影响 57:24
 - **典型误差**：量化损失约9-10%，表现为细节信息丢失(如诺贝尔奖得主名称错误)。
 - **任务差异**：数值计算、长文档定位等精度敏感任务受影响较大。
 - **硬件加速**：低精度计算在专用硬件上可获得显著速度提升，适合实时性要求高的场景。

7) 量化的定义与方法 59:47

量化的定义与方法



任务类型	FP16 vs. 4-bit 差异示例	错误表现
① 数值计算	问：“123456×789012 等于多少？”	FP16 全对； 4-bit 把中间步骤的进位算错，结果差 3~4 位
② 闭卷知识问答	问：“2022 年诺贝尔化学奖三位得主分别是谁？”	FP16 精准列出 Carolyn Bertozzi 等三人； 4-bit 漏掉一人或名字拼写错误
③ 长文本定位	给一篇 4k token 法律合同，问第 2 页第 3 条违约责任金额	FP16 能定位“在第 2 页第 3 条，违约金为合同总价的 5%”； 4-bit 给出 3% 这类相近却错误的数字
④ 少样本分类	5-shot 新闻分类（体育/财经/科技）	FP16 准确率 94%； 4-bit 掉到 78%，把“财经融资”错标成“科技”
⑤ 创意写作	写一首五言绝句，要求押“春”韵	FP16 押韵且意境连贯； 4-bit 出现“春/唇/醇”混押或平仄失调

量化越低，模型越小、越快，但越可能在需要精细数值推理或知识细节的任务上出错。

- 量化越低，模型越小、越快，但越可能在需要精细数值推理或知识细节的任务上出错。
 - **FP16与4-bit差异：**
 - **数值计算：**FP16能准确计算 123456×789012 等大数运算，4-bit会因中间步骤进位错误导致结果偏差3-4位
 - **知识问答：**FP16可完整列出2022年诺贝尔化学奖三位得主（如Carolyn Bertozzi等），4-bit会出现漏人或拼写错误
 - **文本定位：**在4k token法律合同中，FP16能精确定位“第2页第3条违约金为合同总价5%”，4-bit会给出3%等错误数值
 - **少样本分类：**5-shot新闻分类任务中，FP16准确率94%，4-bit降至78%，易将“财经融资”误标为“科技”
 - **创意写作：**FP16能完成押“春”韵的五言绝句且意境连贯，4-bit会出现“春/唇/醇”混押或平仄失调
 - **量化规律：**量化位数越低（如4-bit），模型体积越小、推理速度越快，但在需要数值精度或知识细节的任务上错误率显著增加
- 8) 总结 01:00:01
- DeepSeek不同尺寸版本及其特点 01:00:09
 - **型号谱系：**
 - 小尺寸：1.5B（可笔记本部署）、7B/8B（个人使用，需显卡）
 - 中尺寸：14B（数学推理/代码生成）、32B（企业入门级）
 - 大尺寸：70B（需8张H800）、671B满血版（成本200-300万）
 - **性能特点：**
 - 1.5B：无需显卡，内存即可运行
 - 7B：适合中等复杂度任务，需显卡支持
 - 14B：可处理数学推理和代码生成
 - 32B：企业级性价比选择，部署成本约10万元
 - 70B：存在显著**幻觉问题**，部署需8张H800或L40（约52万）
 - DeepSeek模型部署的成本与问题 01:01:48
 - **成本参考：**
 - 32B版本：约10万元
 - 70B版本：8张L40显卡约52万（单卡6.5万）
 - 671B满血版：200-300万元
 - **主要问题：**
 - 70B版本存在思维链过长导致的幻觉现象
 - 模型会将推理过程误认为用户输入上下文
 - 企业反馈实际使用效果不及预期
 - DeepSeek模型与其他模型的对比与选择 01:04:27
 - **替代方案：**
 - 通义千问3系列：8B（个人用）、32B（企业用）、MOE版（激活参数22B）
 - 8B版本可用24G显存的4090显卡部署

- 实际测试8B效果可媲美DeepSeek 671B
- 选择建议：
 - 企业级推荐千问32B或MOE版
 - 知识库问答场景8B已足够
 - 移动端/嵌入式设备需用小尺寸模型
- DeepSeek模型部署建议 01:05:15
 - 硬件配置：
 - 8B：单张4090显卡（24G显存）- *AutoDL 4090 124G*
 - 32B：企业级服务器 - *Dense 模型最高版本*
 - 70B：需8张H800/L40显卡集群
 - 部署策略：
 - 小尺寸模型直接全量部署
 - 大模型建议采用vLLM等推理框架
 - 需考虑**多并发时的冗余设计**
- 多并发问题与量化策略 01:07:03
 - 并发处理：
 - 需要多卡并行和负载均衡
 - 推荐使用vLLM等专用推理框架
 - 量化应用：
 - 企业部署通常直接使用小尺寸全量模型
 - 量化主要用于模型压缩（如175B模型从70G压缩至20G）
 - 常用方法：GPTQ（4-bit量化）、QLoRA（动态量化）

八、导出量化01:08:58

- 模型创新点与尺寸：课程讲解了模型的创新点、尺寸以及中间层量化等内容，帮助建立基本概念框架
- 自主探索建议：建议学员自行查看相关导出量化内容，作为课后延伸学习材料

九、DeepSeek使用01:09:22

1. 例题:小球碰撞HTML页面编写 01:09:48

CASE：物理世界中的小球碰撞

一个红色的小球在三角区域内运动，碰到边界就反弹，编写一个HTML网页

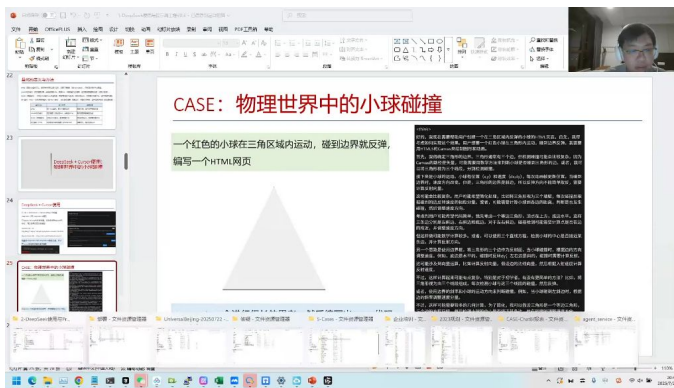


deepseek-r1 会进行很长时间的思考，然后编写出HTML代码。
编写的HTML代码运行可能不符合要求，需要进一步调整

01:09:48



01:09:48



- 实现难点：
 - 边界碰撞检测需要处理斜边反射向量计算
 - 需要考虑重力、弹力和摩擦力的物理模拟
 - 三角形边界处理比矩形更复杂，需要几何计算
- 解决方案：
 - 将三角形视为三个独立线段分别检测碰撞
 - 使用Canvas绘制图形和动画
 - 通过位置 (x,y) 和速度 (dx,dy) 实现运动逻辑
- 代码优化：
 - 增加刷新按钮重新执行HTML
 - 处理中文乱码问题
 - 实现页面自适应布局
- 模型对比：
 - DeepSeek-V3难以完成此类复杂物理计算任务
 - DeepSeek-R1通过长思考过程生成初步解决方案
 - 需要多次迭代修正边界检测等bug

Summary



Thinking: 当我们写程序的时候，什么时候用 DeepSeek-V3-0324，什么时候用 DeepSeek-R1?

V3-0324: 日常编程、快速开发、前端代码生成、常规脚本任务。

R1: 数学密集型计算、复杂算法、代码逻辑深度优化、需要推理过程的任务。

=> 更擅长复杂算法实现，能优化逻辑并减少错误

2. 总结 01:12:26

- 模型选择原则：
 - **V3-0324适用场景**：日常编程、快速开发、前端代码生成、常规脚本任务
 - **R1适用场景**：数学密集型计算、复杂算法、代码逻辑深度优化、需要推理过程的任务
- 使用技巧：
 - 简单任务避免使用R1防止过度思考引入幻觉
 - 复杂任务优先选择R1以获得更好算法实现
 - 出现幻觉时应开启新对话而非持续修正
- 性能对比：
 - R1在代码能力上优于V3但响应速度较慢
 - V3适合绝大多数常规编程任务
 - R1特别擅长物理、数学相关复杂逻辑实现

十、DeepSeek私有化部署01:16:59

1. 部署情况 01:17:07

私有化部署 (DeepSeek模型)



DeepSeek-R1模型

Model	#Total Params	#Activated Params	Context Length	Download
DeepSeek-R1-Zero	671B	37B	128K	https://modelscope.cn/models/deepseek-ai/DeepSeek-R1
DeepSeek-R1	671B	37B	128K	https://modelscope.cn/models/deepseek-ai/DeepSeek-R1-Zero

DeepSeek-R1蒸馏模型

Model	Base Model	Download
DeepSeek-R1-Distill-Qwen-1.5B	Qwen2.5-Math-1.5B	https://modelscope.cn/models/deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B
DeepSeek-R1-Distill-Qwen-7B	Qwen2.5-Math-7B	https://modelscope.cn/models/deepseek-ai/DeepSeek-R1-Distill-Qwen-7B
DeepSeek-R1-Distill-Llama-8B	Llama-3.1-8B	https://modelscope.cn/models/deepseek-ai/DeepSeek-R1-Distill-Qwen-14B
DeepSeek-R1-Distill-Qwen-14B	Qwen2.5-14B	https://modelscope.cn/models/deepseek-ai/DeepSeek-R1-Distill-Qwen-14B
DeepSeek-R1-Distill-Qwen-32B	Qwen2.5-32B	https://modelscope.cn/models/deepseek-ai/DeepSeek-R1-Distill-Qwen-32B
DeepSeek-R1-Distill-Llama-70B	Llama-3.3-70B-Instruct	https://modelscope.cn/models/deepseek-ai/DeepSeek-R1-Distill-Llama-70B

-
- 模型选择：
 - DeepSeek-R1-Zero：总参数量671B，激活参数量37B，上下文长度128K
 - DeepSeek-R1：总参数量671B，激活参数量37B，上下文长度128K
- 蒸馏模型：
 - 基于不同基础模型(Qwen/Llama)的多个版本，参数量从1.5B到70B不等
- 部署建议：
 - 个人使用：推荐1.5B或7B版本，1.5B可在无GPU环境下运行(文件大小约4G)
 - 企业级应用：可考虑32B或更大版本
- 模型获取：
 - 通过ModelScope平台检索下载(网址：modelscope.cn)
 - 示例检索关键词："deepseek-r1"

2. vllm使用 01:19:04

vllm使用



vllm使用：是由伯克利大学 LMSYS 组织开源的LLM高速推理框架，用于提升LLM的吞吐量与内存使用效率。它通过 PagedAttention 技术高效管理注意力键和值的内存，并结合连续批处理技术优化推理性能。vLLM 支持量化技术、分布式推理、与 Hugging Face 模型无缝集成等功能

```
vllm serve deepseek-ai/DeepSeek-R1-Distill-Qwen-32B --tensor-parallel-size 2 --max-model-len 32768 --enforce-eager
```

- vllm serve, 启动 vLLM 推理服务的命令
- deepseek-ai/DeepSeek-R1-Distill-Qwen-32B, Hugging Face 模型库中的模型名称, vLLM 会尝试从 HF 下载模型
- --tensor-parallel-size 2, 启用张量并行, 在 2 个 GPU 上分布式运行模型 (适合 32B 大模型)
- --max-model-len 32768, 设置模型的最大上下文长度 (32K tokens), 确保能处理长文本。
- --enforce-eager, 禁用 CUDA Graph 优化 (可能在某些环境下更稳定, 但性能稍低)

-
- 框架特点：
 - 由伯克利大学LMSYS组织开源
 - 通过PagedAttention技术高效管理注意力键值内存
 - 支持连续批处理技术优化推理性能
 - 支持量化技术、分布式推理、与Hugging Face模型无缝集成
- 适用场景：
 - 需要高效多用户/多显卡的生产环境部署
 - 大模型推理优化

3. vllm服务配置示例 01:20:18

Vllm使用



Thinking: 如果我在本地的ubuntu下面有 /root/autodl-tmp/models/tcf90/deepseek-r1-distill-qwen-32b-gptq-int4, 如何使用vllm进行推理?

```
vllm serve /root/autodl-tmp/models/tcf90/deepseek-r1-distill-qwen-32b-gptq-int4 --tensor-parallel-size 1 --max-model-len 32768 --enforce-eager --quantization gptq --dtype half
```

关键改动: 指定本地路径: 替换 HF 模型名为你的本地路径。

--quantization gptq: 显式声明使用 GPTQ 量化。

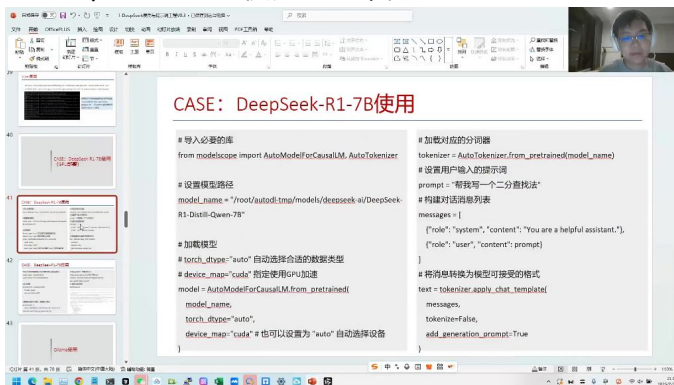
--dtype: 设为 half (FP16) 或 auto (自动选择), 因为 GPTQ 本身是 4-bit, 但计算时需指定中间精度。

```
Loading safetensors checkpoint shards: 0% Completed 0/4 [00:00:?, 911/s]
Loading safetensors checkpoint shards: 25% Completed 1/4 [00:00:00:02, 1.131/s]
Loading safetensors checkpoint shards: 50% Completed 2/4 [00:00:00:02, 1.226/s]
Loading safetensors checkpoint shards: 75% Completed 3/4 [00:03:00:01, 1.226/s]
Loading safetensors checkpoint shards: 100% Completed 4/4 [00:04:00:00, 1.176/s]
Loading safetensors checkpoint shards: 100% Completed 4/4 [00:04:00:00, 1.176/s]
ERROR 09-25 11:02:05 (core.py:340) RuntimeError: Out of memory: Occurred when warming up sampler
due to excessive max_seq_len or max_memory_utilization when initializing the engine.
ERROR 09-25 11:02:05 (core.py:340) Got fatal signal from worker process, shutting down.
```

-
-
- 基本命令结构:
- 关键参数说明:
 - --tensor-parallel-size: 设置GPU并行数量(如2表示使用2个GPU)
 - --max-model-len: 设置最大上下文长度(如32768表示32K tokens)
 - --enforce-eager: 禁用CUDA Graph优化(稳定性更高但性能稍低)
 - --quantization: 指定量化方式(如gptq)
 - --dtype: 设置计算精度(如half表示FP16)
- 硬件要求:
 - 32B模型即使在GPTQ量化(int4)后仍需要超过24GB显存
 - 7B模型更适合个人GPU(如RTX 4090)部署

AutoDL算力云 <https://www.autodl.com/home>

4. CASE DeepSeek-R1-7B使用GPU部署 01:22:14



-
- 部署流程:
 - 导入必要库:
 - 设置模型路径:
 - 加载模型和分词器:
 - 构建对话消息:
 - 生成响应:
- 注意事项:
 - 7B模型文件大小约15G, 需要GPU支持
 - 推理过程中会显示详细的思考过程(草稿纸式推演)
 - 响应生成速度取决于GPU性能

十一、代码下载与模型部署

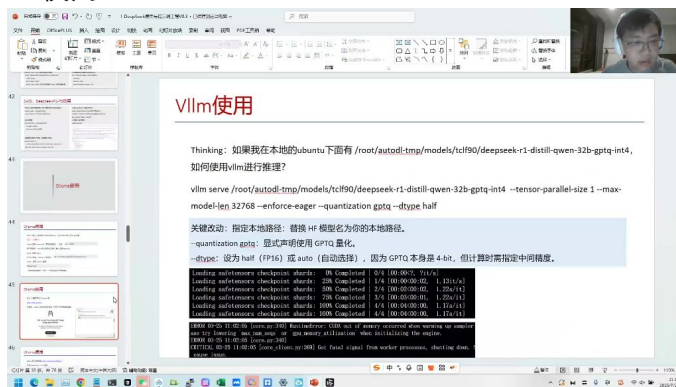
- 模型下载方法: 通过modelscope包从模型库下载DeepSeek-R1-7B模型
- 存储路径: 下载到/root/autodl-tmp/models/deepseek-ai/目录下
- 文件大小: 模型文件总大小约15GB
- 下载优势: 国内直连下载速度非常快

1. 模型部署方法

- 方法一：使用vrom进行部署
 - 方法二：在焚框架内调用部署好的模型
 - 方法三：使用Ollama（最简单的方法）
- ## 2. 模型ID获取
- 获取方式：在模型页面直接复制模型ID
 - 作用：模型ID代表模型身份，用于从modelscope网站自动下载对应文件
- ## 3. vLLM框架使用
- 框架特点：
 - 由伯克利大学LMSYS组织开源
 - 用于提升LLM的吞吐量与内存使用效率
 - 通过PagedAttention技术管理注意力键和值的内存
 - 结合连续批处理技术优化推理性能
 - 支持功能：
 - 量化技术
 - 分布式推理
 - 与Hugging Face模型无缝集成
 - 启动命令：
 - 参数说明：
 - --tensor-parallel-size 2：在2个GPU上分布式运行模型
 - --max-model-len 32768：设置模型最大上下文长度为32K tokens
 - --enforce-eager：禁用CUDA Graph优化（在某些环境下更稳定）
 - 硬件要求：推理大尺寸模型一般需要GPU支持，可以考虑租用GPU资源

十二、DeepSeek使用与提示词工程01:30:00

1. vLLM使用 01:30:39



- 本地路径指定：需将HF模型名替换为本地实际路径，格式为\$V\parallel m s e r v e /path/to/local/model\$
- 量化参数：
 - 显式声明GPTQ量化：-quantization gptq
 - 计算精度设置：-dtype half (FP16) 或auto（自动选择），GPTQ本身为4-bit但需指定中间精度
- 常见错误：可能出现CUDA out of memory显存不足报错，需调整-tensor-parallel-size等参数

2. Ollama使用 01:31:19

1) 安装方法一（WSL）

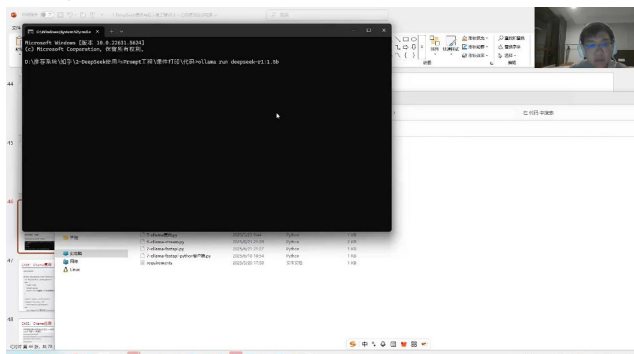
- 步骤1：管理员权限运行wsl --install，重启后自动安装Ubuntu
- 步骤2：在WSL终端执行安装命令：\$curl -fsSL https://ollama.com/install.sh | sh\$
- 步骤3：启动服务ollama serve（需保持终端运行）

2) 安装方法二（原生） 01:31:45

- 适用系统：官方支持macOS/Linux，Windows可通过模拟环境运行
- 优势：相比WSL方案更简单直接，适合快速部署
- 模型管理：安装后可通过命令行拉取/删除模型（如ollama pull deepseek-r1:1.5b）

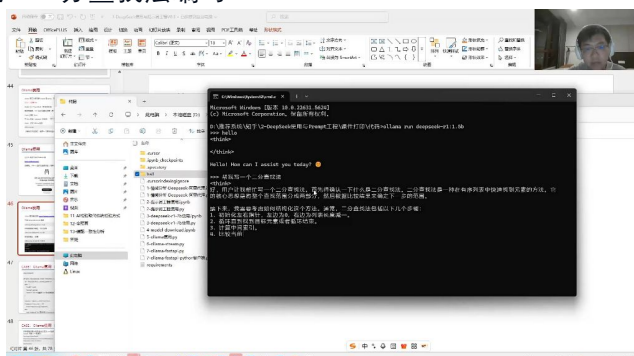
3) 应用案例 01:32:48

- 模型交互演示



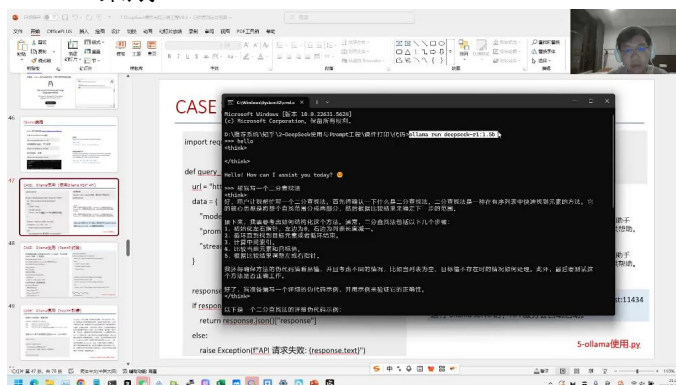
- 启动命令：ollama run deepseek-r1:1.5b进入交互界面
- 性能特点：1.5B小模型在CPU上推理速度较快（约1.1GB下载大小）
- 文件存储：自动下载模型至Ollama管理文件夹

- 例题：二分查找法编写



- 实现步骤：
 - 初始化左右指针（左=0，右=列表长度-1）
 - 循环直到找到目标或范围耗尽
 - 计算中间索引：mid = (left + right) // 2
 - 比较中间元素与目标值
 - 动态调整左右边界
- 异常处理：需考虑空列表、目标不存在等边界情况

4) REST API集成



- 基础配置：

- 默认端点: `http://localhost:11434/api/generate`
- 必需参数: `model` (模型ID)、`prompt` (输入文本)
- 流式响应: 设置`"stream": true`获取实时输出

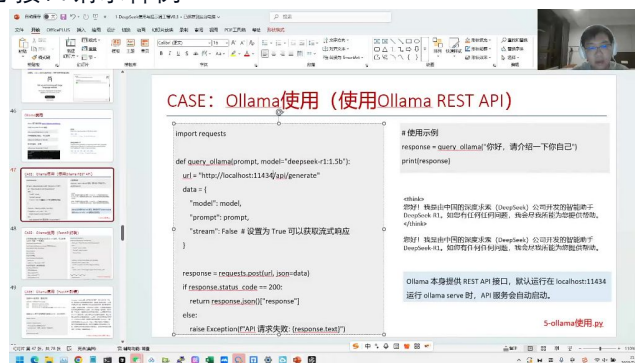
● Python封装:

```
1 def query_ollama(prompt, model="deepseek-r1:1.5b"):
2     response = requests.post(url, json={
3         "model": model,
4         "prompt": prompt,
5         "stream": False
6     })
7     return response.json()["response"]
```

3. Ollama使用—REST API接口 01:34:14

1) 接口封装示例

- 默认端口: Ollama REST API默认运行在`localhost:11434`端口, 运行`ollama serve`时API服务会自动启动
- 基础请求方法:
- 流式响应: 当`stream`参数设为`True`时, 响应会逐字返回, 类似聊天对话体验; 设为`False`则需等待完整响应
- 模型选择: 个人使用建议1.5b小模型, 企业级建议32b以上大模型
- 例题:接口请求样例

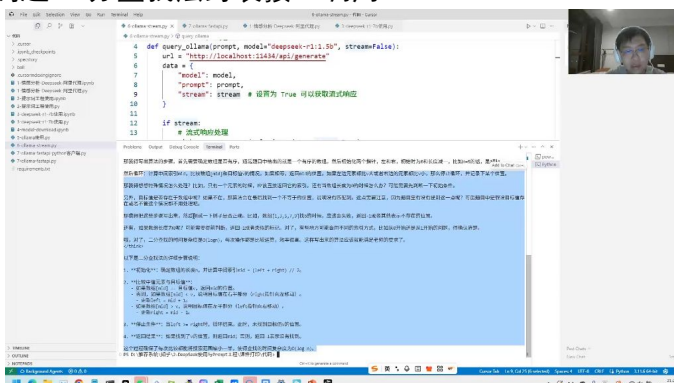


○ 题目解析

- 典型请求: `response = query_ollama("你好, 请介绍一下你自己")`
- 预期响应: 包含模型自我介绍的标准JSON格式响应
- 调试要点: 检查端口11434是否开放, 模型名称是否拼写正确

4. 应用案例 01:35:28

1) 例题:二分查找法封装接口调用



● 题目解析

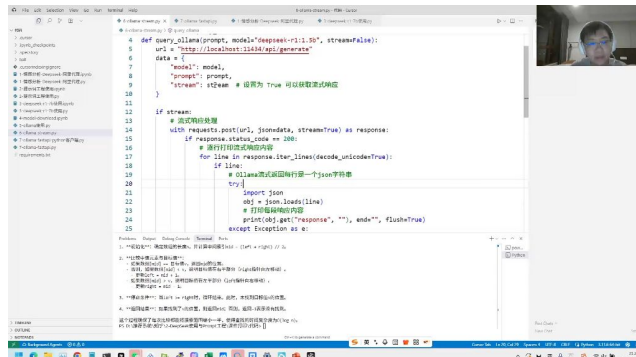
- 算法步骤:
 - 初始化左右指针(`left=0, right=len(arr)-1`)
 - 循环计算中间索引`mid=(left+right)//2`

- 比较arr[mid]与目标值：
 - 相等则返回mid
 - 小于目标值则调整left=mid+1
 - 大于目标值则调整right=mid-1
- 循环终止条件：left>=right

○ 特殊处理：

- 空数组直接返回-1
- 单元素数组直接比较
- 目标不存在时的返回约定

- 时间复杂度： $O(\log n)$ ，每次比较将搜索范围减半
- 实现验证：建议用有序数组[1,3,5,7,9]测试边界情况



● 进阶封装：

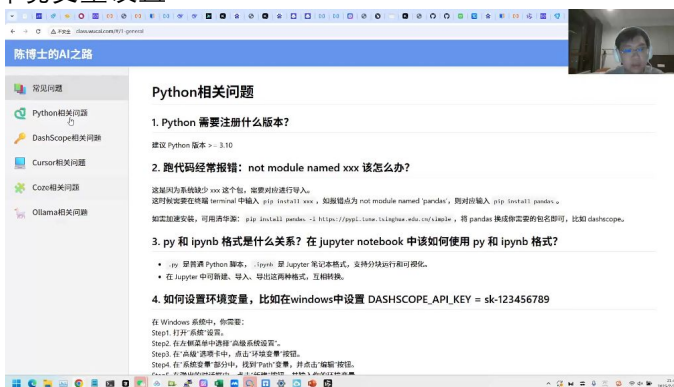
- 使用FastAPI或Flask框架将接口封装为Web服务
- 可自定义服务端端口(如8000/8001)
- 安装依赖：pip install fastapi uvicorn
- 优势：提供标准化API接口供其他系统调用

● 调试建议：

- 首次运行需处理ModuleNotFoundError
- 小模型(1.5b)响应速度较快，适合本地测试
- 企业级应用建议采用vLLM等专业部署方案

5. 常见问题 01:42:00

1) 环境变量设置



● 设置步骤：

- Step1.打开"系统"设置
- Step2.在左侧菜单中选择"高级系统设置"
- Step3.在"高级"选项卡中，点击"环境变量"按钮
- Step4.在"系统变量"部分中，找到"Path"变量并点击"编辑"

● Python版本建议：建议使用Python版本>=3.10

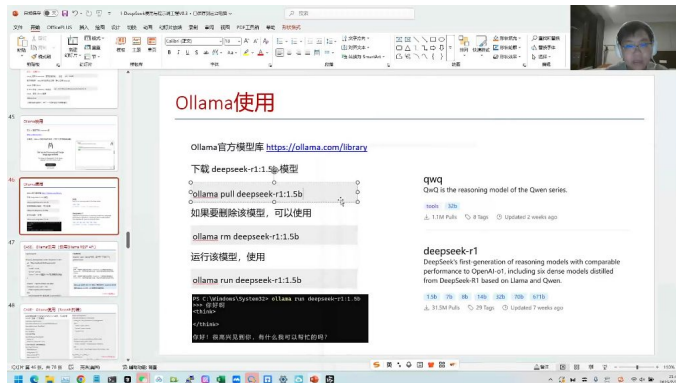
● 模块缺失处理：

- 当出现"not module named xxx"错误时，需要通过终端执行pip install xxx安装缺失包
- 可使用清华源加速安装：pip install pandas -i https://pypi.tuna.tsinghua.edu.cn/simple

● 文件格式区别：

- .py是普通Python脚本
- .ipynb是Jupyter笔记本格式，支持分块运行和可视化
- 在Jupyter中可互相转换这两种格式

2) Ollama相关问题 01:42:49



● 基本使用：

- 下载模型：ollama pull deepseek-r1:1.5b
- 运行模型：ollama run deepseek-r1:1.5b
- 删除模型：ollama rm deepseek-r1:1.5b

● 路径修改：

- 创建新目录如
- 设置环境变量OLLAMA_MODELS值为新路径
- 迁移旧模型文件并重启服务

● 系统要求：

- CPU:双核或以上(x86_64架构)
- 内存:至少4GB(运行3B~7B模型)
- GPU:非必须（仅用于模型文件）

● 模型特点：

- 1.5b和7b模型性能相当
- 可量化到更小尺寸如0.6b
- 适合少量用户使用，并发性能有限

3) Fast API和vLLM相关问题 01:43:51

● FastAPI作用：

- 用于构建API接口
- 封装后端服务接口

● vLLM特点：

- 高速推理大模型工具
- 底层优化包括page attention管理
- 支持多GPU部署和上下文长度设置
- 适合高性能部署场景

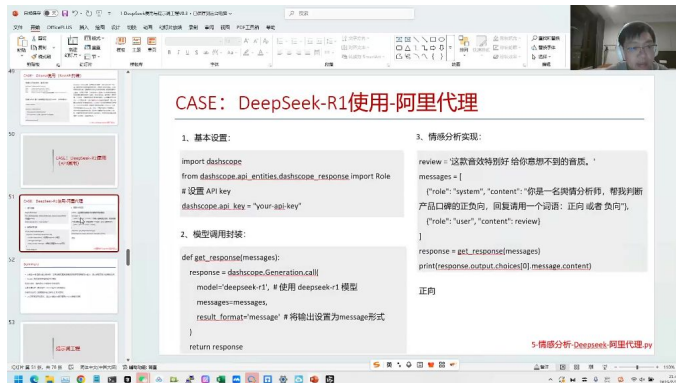
● 部署方式：

- 多显卡配置：支持一机八卡并行
- 分布式部署：需根据硬件配置策略
- 启动指令：vllm --model 模型地址

- 私有化部署限制：
 - 模型尺寸受限
 - 上网环境可能受限
 - 系统集成复杂度高

6. 提示工程使用示例 01:45:55

1) 例题:产品口碑判断 01:48:19



- 实现步骤：
 - 设置API key: `dashscope.api_key = "your-api-key"`
 - 封装模型调用函数
 - 构建消息列表包含系统提示和用户评价
 - 调用模型获取响应
- 代码要点：
 - 使用`dashscope.Generation.call`调用模型
 - 指定模型为`deepseek-r1`
 - 设置结果格式为`message`
 - 从响应中提取输出内容
- 应用场景：
 - 舆情分析
 - 产品评价分类
 - 情感倾向判断
- 输出格式：
 - 限定输出为"正向"或"负向"单一词语
 - 通过`response.output.choices[0].message.content`获取结果

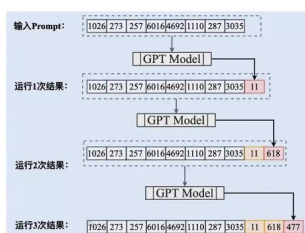
7. 提示词工程原理 01:56:48

Prompt原理

Prompt原理：

- GPT在处理Prompt时，GPT模型将输入的文本（也就是Prompt）转换为一系列的词向量。
- 然后，模型通过自回归生成过程逐个生成回答中的词汇。在生成每个词时，模型会基于输入的Prompt以及前面生成的所有词来进行预测。
- 这个过程不断重复，直到模型生成完整的回答

Thinking: 你认为Prompt Engineering的重要性如何?



- 向量转换过程：GPT模型将输入的Prompt文本转换为一系列词向量，作为模型理解的数字表示形式。
- 自回归生成机制：模型通过逐个生成词汇的方式构建回答，每个新词的生成都基于完整Prompt和已生成的全部内容。

- **迭代预测特性：**生成过程持续循环直至输出终止符，每次迭代都会重新评估上下文关系（例如示例中从"11"到"618"再到"477"的递进生成）。
- **核心价值：**
 - **质量决定性：**提示词直接决定大模型输出的内容质量，是模型理解用户意图的第一手资料。
 - **效率杠杆：**优化提示词可显著减少交互次数，提升任务完成效率达30-50%。
 - **歧义消除：**精确的提示词能降低模型误解概率，特别是在专业领域应用中效果显著。

8. 提示词策略差异 01:57:52

1) 模型类型对比

- **推理模型特征：**
 - **自主延展性：**具备自动推导能力（如物理实验模拟），只需提供问题核心即可
 - **提示词简洁原则：**类似"计算小球在三角体中的重力轨迹"的简洁指令即可触发复杂推理
 - **适用场景：**适合需要逻辑推导的复杂任务（示例中的三角体重力实验）
- **通用模型特征：**
 - **显性指导需求：**必须明确步骤指令，如"第一步分析材料，第二步比较数据"
 - **补充式设计：**采用"缺什么补什么"策略，需完整交代背景信息
 - **适用场景：**创意类任务（情书写作）和简单问答场景

2) 选择方法论

- **任务导向原则：**
 - **复杂度评估：**物理模拟等需要逐步推导的任务优先选择推理模型
 - **创意度评估：**文学创作等发散性任务更适合通用模型
 - **反例警示：**用推理模型写情书会导致过度逻辑化（出现数学公式等不适宜内容）
- **设计准则：**
 - **推理模型三不要：**不要过度简化、不要步骤分解、不要限制输出格式
 - **通用模型三要素：**明确步骤、补充背景、预设输出结构
 - **黄金法则：**模型选择应完全取决于任务本质，而非市场热度

9. 提示词编写原则 02:00:52

提示词策略差异



通用模型	推理模型
• 需要显示引导推理步骤，比如通过CoT提示，否则可能会忽略关键逻辑 • 依赖提示词补偿能力短板，比如要求分步骤思考，提供few-shot参考示例等	• 提示语更简洁，指需要明确任务目标和需求，因为模型已经内化了推理逻辑 • 无需逐步指导，模型会自动生成结构化推理过程。如果强行分步骤拆解，反而会降低其推理能力

- ④ ⑤ ⑥ ⑦ ⑧ ⑨ ⑩ ⑪ ⑫ ⑬ ⑭ ⑮ ⑯ ⑰ ⑱ ⑲ ⑳ ㉑ ㉒ ㉓ ㉔ ㉕ ㉖ ㉗ ㉘ ㉙ ㉚ ㉛ ㉜ ㉝ ㉞ ㉟ ㊱ ㊲ ㊳ ㊴ ㊵ ㊶ ㊷ ㊸ ㊹ ㊺ ㊻ ㊼ ㊽ ㊾ ㊿

1) 具体指导

- **任务明确性：**当要求AI完成摘要写作时，模糊指令如"请生成摘要"会导致AI依赖训练数据的随机格式，而具体指令如"用3-4句话概括主要事件、人物、时间地点"能获得更符合预期的输出
- **细节控制：**在订单咨询场景中，简单指令"回答订单问题"与详细指令"礼貌回答并提供具体订单信息、预计到达时间及解决方案"会产生显著不同的回答质量
- **视觉创作指导：**以"生成美女图片"为例，不具体描述会导致输出受训练数据主导（如欧美模特风格），而详细说明发型、画风、背景等特征能精确控制输出效果

2) 简洁明了 02:02:24

- **信息密度平衡**：虽然需要具体指导，但应避免冗余描述。例如故事续写任务中，简短指令"根据开头续写故事"与冗长说明"保持风格一致、注意连贯性..."效果相当，后者可能稀释关键信息
- **最小必要原则**：能用一句话清晰表达就不使用多句，确保每个词都有信息价值。过度解释会干扰AI对核心要求的注意力
- **矛盾处理**：具体与简洁并非对立，而是要在保证关键约束的前提下剔除无实质影响的修饰语

3) 适当引导 02:04:06

- **示例教学法**：提供模板或样例（如请假邮件模板）能让AI快速掌握格式要求，且能自动适配新内容（识别示例中的日期等变量）
- **历史资料利用**：不需要专门制作示例，可直接使用过往工作记录作为参考，相当于为AI建立"工作手册"
- **人机交互视角**：将AI视为实习生，通过提供历史案例库（如各种办公文档模板）培养其完成任务的能力
- **迭代优化机制**：接受AI输出的初稿不完美，通过多次反馈调整逐步完善，模拟人类学习过程

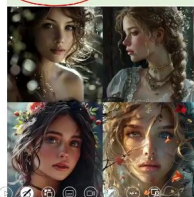
Prompt 编写原则

● **具体指导**：给予模型明确的指导和约束。

示例 3：生成 a beautiful girl 的图片

不明确的指导：

a beautiful girl



具体的指导：

anime girl with long dark hair , simple and elegant style, light silver and light gray, wavy resin sheets , in the style of digital painting, gongbi, realistic yet romantic, shiny/glossy, cute and dreamy, smooth lines, pure white background,

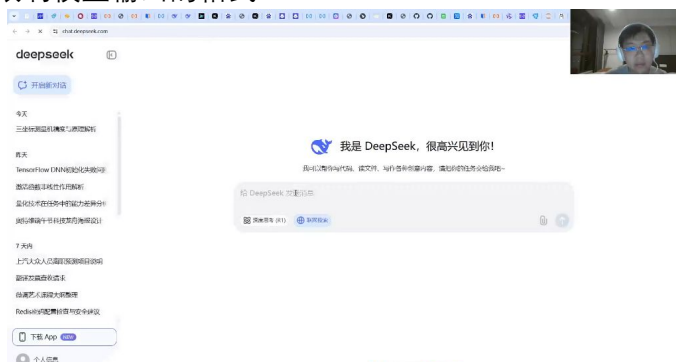


动漫少女，长黑发
数码画风，纯白底色

-
- **结构化设计**：有效提示词应包含任务目标、背景信息、参考示例、受众角色、格式要求和语气风格六个要素
- **优先级排序**：明确任务指令最核心，其次是补充背景，最后是格式语气等修饰性要求
- **动态调整**：提示词编写是持续优化的过程，需要根据输出结果不断修正各要素的表述方式

10. 提示词编写技巧 02:08:45

1) 限制模型输出的格式 02:09:36



-
- **结构化输出优势**：要求大模型输出JSON格式可以结构化数据，方便后续解析。例如从财务报表中提取信息时，JSON格式能清晰呈现"公司名称"、"股票代码"等字段的对应关系。
- **非结构化对比**：若不用JSON格式，模型输出可能是冗长段落，需要人工二次提取数据。如伊利集团案例中，JSON格式直接呈现了"营收325亿元"等关键数据。

- **应用场景：**特别适用于从PDF等非结构化文档中提取结构化信息，如财务报表分析、数据报表生成等任务。

2) 使用分隔符区分输入的不同部分 02:10:59



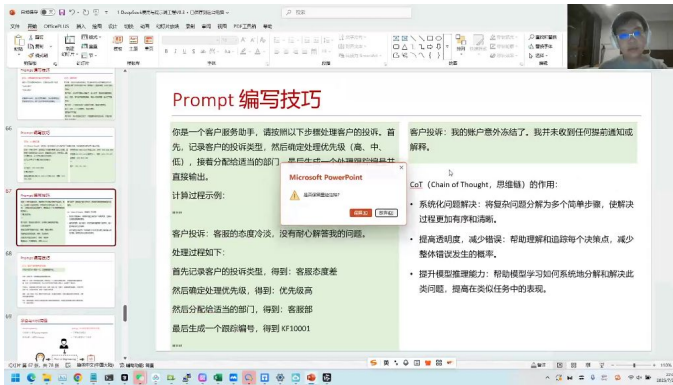
- **分隔符作用：**用三个引号等符号分隔prompt中的指令、示例、注意事项等不同模块，帮助模型准确理解各部分内容。
- **自定义灵活性：**分隔符可自由定义，如三个等号、四个井号、五个星号等，只需保持人眼可识别即可，AI就能理解其分隔意图。
- **替代方案：**除三个引号外，也可使用逗号、换行符、括号等作为分隔符，关键是要有明确的区隔意识。
- **典型应用：**在复杂prompt中区分：指令说明、输入数据、输出要求、易错点提示等不同功能模块。

3) 提供样例 02:12:30

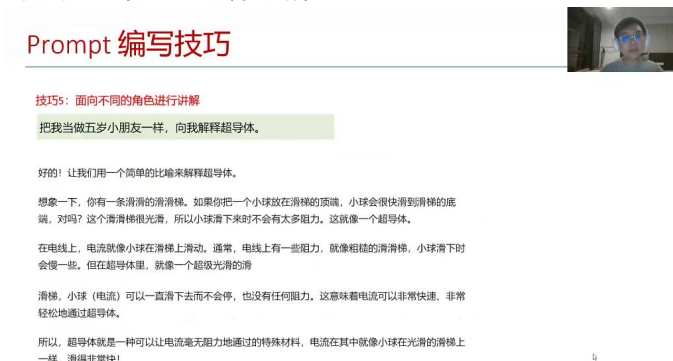


- **适用场景：**
 - 处理反语或特殊表达时 (如"这次开户真是太满意了"实际表达不满)
 - 解释行业术语时 (如"黑粉"、"白粉"等企业特定术语)
 - 规范复杂输出格式时
- **管培生类比：**将大模型视为管培生，通过提供完整样例（如完整计算过程），模型能模仿该模式处理新任务。
- **舆情分析案例：**当用户评价包含反语时，提供明确的正负面样例可显著提高分类准确率。

4) CoT思维链工具 02:14:06



- 核心价值：
 - 系统化分解复杂问题为多个简单步骤
 - 提高解决过程的透明度和可追溯性
 - 降低整体错误发生率
 - 增强模型在同类任务中的推理能力
 - 数学计算示例：分步演示"先加1000，再减500，最后乘1.2"的计算过程，模型能准确复现该逻辑处理新输入。
 - 客服流程案例：明确"记录投诉类型→确定优先级→分配部门→生成编号"的标准化流程，确保处理一致性。
 - 业务逻辑传承：在金融等专业领域，通过思维链可让模型学习业务方的专业处理逻辑，如账户冻结等敏感事件的标准应对流程。
- 5) 面向不同的角色进行讲解 02:18:46



- 受众适配：根据听众认知水平调整解释方式，如向五岁儿童解释超导体时，使用滑梯和小球的生活化比喻。
 - 比喻技巧：
 - 将电流比作滑梯上的小球
 - 常规导体=粗糙滑梯（有阻力）
 - 超导体=超级光滑滑梯（无阻力）
 - 专业转换：把"电流无阻力通过"的专业表述转化为"小球可以一直滑下去不会停"的儿童语言
 - 跨领域应用：适用于教育、客服、产品说明等需要差异化表达的场景
11. 实战案例 02:20:26
- 1) 例题：手机流量套餐选择需求分析 02:22:24
- 任务设计：通过提示词让AI识别用户对手机流量套餐的选择需求，套餐包含名称、月费、月流量三个属性
 - 实现方法：
 - 提示词结构：分为系统角色定义（AI目标）和用户输入两部分

- **输出优化**：添加JSON格式约束可使输出结构化，例如用户输入"办一个100g的套餐"时，输出仅包含有效字段：{"名称":"","月费":"","月流量":"100G"}
- **模型视角**：大模型实际处理的是拼接后的完整提示词，包含角色定义和用户输入
- 2) 例题：客服回答规范判断 02:25:00
- **规范要求**：
 - 必须使用官方口吻（禁用网络用语如"亲"）
 - 必须完整提及套餐三要素：名称、月费、月流量
- **判断方法**：
 - **分步分析指令**：添加"请一步一步分析对话"可显著提升模型推理能力
 - **检查维度**：礼貌性检查→官方口吻检查→信息完整性检查
- **示例分析**：客服回答"亲，我们正在推广无线套餐..."被判为不规范，因：
 - 使用非官方用语"亲"
 - 虽提及三要素但口吻不合规
- 3) 例题：提示词优化 02:27:49
- **优化方法**：
 - **角色设定**：指定AI作为"专业提示词创作者"
 - **优化维度**：提供改进建议、批判性意见、用户问题示例（不超过3个）
- **优化效果**：
 - 改进后的客服提示词包含：热情介绍、明确服务范围、主动询问需求
 - 生成典型问题示例：
 - "您平均每月使用多少流量？"
 - "您的预算范围是多少？"
 - "请问您是在校生吗？"
- **核心技巧**：通过让AI模拟专家角色，自动完成提示词迭代优化
- 4) 大模型部署实践
- **部署要点**：
 - **硬件要求**：7B模型约需24G显存，32B模型需至少4张4090显卡
 - **部署方式**：
 - 企业环境推荐Linux服务器部署
 - 通过FastAPI封装本地端口（如8000）提供API服务
- **调试技巧**：
 - 当输出不符预期时，可要求模型"分步思考"或反问"需要哪些补充信息"
 - 量化技术主要用于低显存场景（如4G显存），常规情况建议直接使用小尺寸模型
- 5) 提示词工程要点
- **本质理解**：提示词即与AI交流的指令（prompt），包含：
 - 系统预设角色
 - 用户具体问题
- **工程师职责**：
 - 设计基础系统提示词框架
 - 优化用户与AI的交互流程
- **安全考虑**：国内企业部署需注意：
 - 优先选择国产化部署方案（如深度求索）
 - 避免直接使用国外开源框架（如vLLM）

十三、大模型本地部署与微调

- **小模型微调可行性**：本地部署小尺寸大模型后，仍可继续训练模型学习专业知识库，二者不冲突。可通过微调生成新参数后重新部署使用。
- **微调技术实现**：针对已部署的小尺寸模型进行微调（fine-tuning），会生成新的模型参数，这些参数可以重新部署到生产环境中使用。

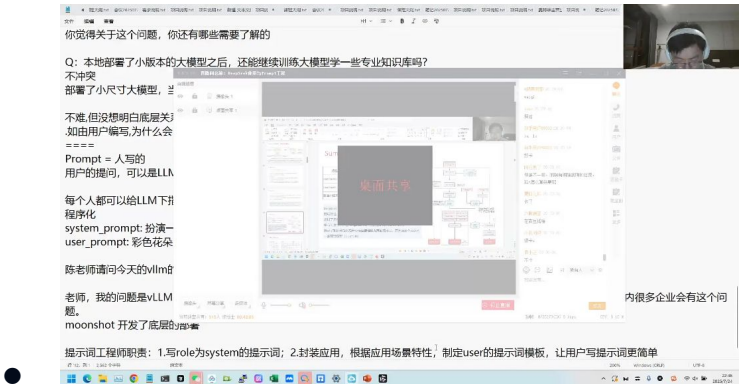
1. 提示词工程体系

- **prompt编写主体**：虽然所有用户都可以编写prompt，但专业场景需要提示词工程师进行系统化设计。
- **工程师核心职责**：
 - 编写system角色提示词（如："扮演公司命名助手"）
 - 封装应用时设计user提示词模板，降低用户使用门槛
- **人设实现原理**：通过LLM内置的system_prompt实现角色设定（如示例中的"公司命名助手"）， user_prompt则接收具体指令（如"彩色花朵"）。

2. 本地化部署实践

- **vLLM安全性质疑**：作为伯克利大学开发的产品，需评估是否满足国内企业信息安全审计要求。
- **国产替代方案**：moonshot等国内厂商已开发底层部署方案，可作为技术备选。
- **实践建议**：非实操情况下，仍需理解vLLM的架构原理对本地化部署的影响。

3. 课后实践任务



- **基础任务**：
 - 完成Ollama本地部署
 - 跑通基础prompt流程（无代码经验者只需完成流程验证）
- **进阶任务**：有开发经验者可尝试prompt模板设计与优化

十四、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
DeepSeek模型创新	MLA架构（5%空间缓存98%信息）、MOE混合专家架构（分诊台机制）、强化学习（自我对弈生成数据）	MLA与MOE的区别、强化学习与传统监督学习的对比	★★★★
模型部署方案	欧拉玛（个人轻量级）、vLLM（企业高性能）、FastAPI封装	7B/32B模型显存需求对比、量化策略选择	★★★
提示词工程原则	具体指导（明确输出格式）、简洁明了（避免信息稀释）、示例引导（提供参照模板）	推理模型与通用模型的提示词差异	★★
量化技术	32位→8位精度压缩（节省75%空间）、混合精度动态调整	GPTQ量化与QLoRA微调的区别	★★★★
模型尺寸选择	1.5B（端侧）、7B（个人）、32B（企业入门）、70B（高精度）	70B模型部署成本估算（52万/8张L40）	★★★

私有化部署实践	ModelScope模型下载、环境变量配置、多GPU并行策略	WSL环境兼容性问题、内网API安全隔离	★★★★
代码生成案例	物理引擎开发（重力/弹力模拟）、二分查找算法实现	推理模型产生的冗余思维链处理	★★★
行业应用场景	金融客服（JSON格式输出）、舆情分析（正负面分类）、教育科普（儿童化表达）	领域黑话术语处理	★★